Deployment Guide

River Hawk Consulting, LLC · IMAX HCD Prototype OT

August 7, 2026

CONTENTS

Deployment Guide — IMAX Chat-Viewer (the target environment packaging)

UNCLASSIFIED — Deliver-deployable set per the target environment container guidance: six images, declarative Kubernetes manifests, no secrets, stateless throughout. Validated on a kind cluster in CI on every push (`package-validate` job); the same steps apply verbatim to the target-environment sandbox.

> **Need every connection to be TLS?** `kubectl apply -k deploy/overlays/tls` — same images, certificate mounted from a Secret, all six pods on https, probes switched to HTTPS, and the SPA edge dialling the enrichment pods over TLS. See `AIRGAP.md` §10.
>
> **Deploying into the air-gapped enclave? Read `AIRGAP.md` first.** This guide assumes a network. The enclave differs in four ways that each stop a deployment cold: images cannot be pulled from public registries, bare image names resolve to Docker Hub and hang, the model gateway's TLS is issued by an internal CA that Node does not trust, and nothing in the SPA may fetch from the internet at runtime. `AIRGAP.md` covers all four, plus the transfer bundle and in-enclave triage.

1. Image inventory

Image	Contents	Base	Port
`imax-spa`	Angular production bundle + Express edge (static, proxy, identity)	`node:22-alpine`	8443
`imax-mock-search`	`routes/search.js` + seed corpus	`node:22-alpine`	5177
`imax-mock-translation`	`routes/translation.js`	`node:22-alpine`	5177
`imax-mock-entities`	`routes/entities.js`	`node:22-alpine`	5177
`imax-mock-summarize`	`routes/summarize.js`	`node:22-alpine`	5177
`imax-mock-ocr`	`routes/ocr.js` + `/static` binary fixtures	`node:22-alpine`	5177

Build all six (from the repo root):

```
./deploy/build-images.sh 0.1.0
```

Every image: multi-stage build, dependencies installed once at build time, non-root user, `.dockerignore`-pruned context, OCI labels, configuration by environment variable only.

Base images are pinned **by digest in the Dockerfiles**, not merely recorded here, so the bits trivy clears are the bits that ship; final stages run `apk upgrade` so OS packages stay current within the pinned base. Re-resolve with `./scripts/refresh-base-digests.sh` (uses `crane`, no Docker daemon needed) and commit the result.

Base	Digest (resolved 1 Aug 2026)
`node:22-alpine`	`sha256:c610fcdfb1d5b4740dd70c284ed3cb16bb857e0f7166196e36a5501df7a3aa32`

One base image for all six. The SPA pod used to run nginx; it now runs an Express edge (`deploy/spa-entry.js`) on the same Node base as the enrichment pods. nginx was serving static files, answering `/healthz` and proxying six prefixes — 25 lines of behaviour in exchange for a second base image, a second package ecosystem to patch and scan, and a second base for the enclave's yard to approve.

Both Dockerfiles take `NODE_BASE` / `NPM_REGISTRY` / `OS_PATCH` build args so the same source rebuilds against an approved in-enclave base and mirror without modification — see `AIRGAP.md` §2.

2. Registry push

Tag and push to the target environment registry (placeholder host):

```
REGISTRY=registry.internal.example/imax
for IMG in imax-spa imax-mock-search imax-mock-translation imax-mock-entities imax-
mock-summarize imax-mock-ocr; do
  docker tag "$IMG:0.1.0" "$REGISTRY/$IMG:0.1.0"
  docker push "$REGISTRY/$IMG:0.1.0"
done
```

If a registry prefix is used, set it on the manifests with a kustomize image override or edit the `image:` fields — tags stay versioned, never `latest`.

3. Deploy

```
kubectl apply -k deploy/k8s
kubectl get pods -l part-of=imax-chat-viewer   # expect 6/6 Running, READY 1/1
```

Every Deployment carries liveness + readiness probes (`GET /healthz`), resource requests/limits (100m/128Mi → 250m/256Mi — modest; tune on the target cluster), and the restricted Pod Security Standard fields (`seccompProfile: RuntimeDefault`, `allowPrivilegeEscalation: false`, `capabilities: drop: [ALL]`) that an enforcing namespace requires. `runAsUser` is deliberately unpinned so a namespace assigning its own UID range can do so.

Mock services are ClusterIP and a NetworkPolicy restricts them to traffic from the SPA pod. `imax-spa` is the sole user-facing surface — attach the target environment ingress/route to `svc/imax-spa:8443`, or validate with:

```
kubectl port-forward svc/imax-spa 8443:8443
open http://localhost:8443
```

TLS ON EVERY HOP

Two overlays, same resulting state — they differ only in where the certificate comes from.

A certificate someone has in hand:

```
kubectl create secret generic imax-tls \
  --from-file=tls.crt=<cert>.pem --from-file=tls.key=<key>.pem --from-file=ca.pem=
<ca>.pem
kubectl apply -k deploy/overlays/tls
```

A certificate in AWS Secrets Manager — the target environment path, for an automated cluster where nobody runs `kubectl create secret`:

```
kubectl apply -k deploy/overlays/tls-awssm
kubectl patch configmap imax-tls-source --type=merge -p '{"data":{
  "AWS_SECRETSMANAGER_ENDPOINT":"<regional endpoint>",
  "AWS_STS_ENDPOINT":"<regional STS endpoint>",
  "TLS_CERT_SECRET":"<cert secret>",
  "TLS_KEY_SECRET":"<key secret, or empty if one secret holds both>"}}'
kubectl rollout restart deploy -l part-of=imax-chat-viewer
```

An initContainer in each pod fetches the material at start and stages it on an in-memory volume, so the private key never lands on a node's disk or in a Kubernetes Secret. Both secret layouts (separate or combined) and both payload formats (raw PEM or JSON) are detected. It needs a role allowing `secretsmanager:GetSecretValue` on those secrets — the policy to ask for is in `k8s/serviceaccount.yaml`.

Same images either way. The overlay mounts the certificate in all six pods, sets `UPSTREAM_SCHEME=https` so the SPA edge dials the enrichment pods over TLS, and switches the probes to `scheme: HTTPS`. `TLS_CLIENT_AUTH=require` adds mutual TLS. Details and

troubleshooting in `AIRGAP.md` §10.

4. Live model gateway (the target environment switch)

The enrichment services answer from fixtures by default. In the target enclave they call the Sponsor's own model gateway over the OpenAI-compatible chat-completions wire format. Switching is configuration, not code or a rebuild:

1. **Three values, all supplied at deploy time, none of them in this repository.** Only the key is a secret; the address and the model name are ordinary configuration:

```
kubectl patch configmap imax-model-gateway --type=merge -p '{"data":{
  "MODEL_ENDPOINT":"<gateway base URL, e.g. https://host/v1>",
  "MODEL_NAME":"<a model the gateway publishes>"}}'
kubectl create secret generic imax-model-gateway \
  --from-literal=MODEL_API_KEY='<key>'
kubectl rollout restart deploy -l part-of=imax-chat-viewer
```

`MODEL_ENDPOINT` must include the version path (`https://host/v1`) — the code appends `/chat/completions`. **All three are required**: miss any one and the pods answer from fixtures, which is a green deployment that is quietly not using the gateway at all. The translation and OCR pods say which it is on their first log line:

```
model gateway on — gateway.host:llama-3
model gateway off — fixtures (unset: MODEL_API_KEY)
```

2. `MODEL_NAME` ships as `SET-ME` rather than a real model name, so an unset deployment fails visibly instead of asking a gateway for something it has never published. Choose a vision-capable model if OCR should run live.

3. For live OCR, rasterize the document fixtures once (`node scripts/rasterize-attachments.js`) — vision models take PNG/JPEG, and the demo scans ship as SVG.

Behavior. Translation sends the whole thread as context, not the message alone. Provenance follows the configuration: a gateway translation is labelled `<gateway-host>:<model>` in the gold copy and its export, so an officer can always see which engine answered. **Any failure — unset endpoint, missing key, unreachable gateway, malformed response — falls back to the fixture and logs it.** A model outage degrades the bench to the mock rather than blocking the linguist.

Locally: copy `.env.example`, fill it in, and start the mock server with those variables exported.

5. Statelessness and secrets

- **Stateless by design**: seed data is baked into the images; all officer work-state (dispositions, reviews, notes, tags, clips, thread gold) lives in the browser session per the evaluated demo's performance claim. No volumes, no PVCs.
- **One optional secret**: `imax-model-gateway/MODEL_API_KEY`, consumed by env var from a Kubernetes Secret and required only when the live gateway is switched on (§4). Nothing is baked into an image; with no Secret the services answer from fixtures.
- **Immutable posture**: config changes are a new image tag + `kubectl rollout`, never exec-and-edit.

6. Production cutover

Two independent seams, both configuration:

- **Enrichment quality** — point the services at the enclave model gateway (§4). Same contracts, live model output, honest provenance.
- **Service topology** — the SPA edge maps `/api/*` to the mock Services by cluster DNS name. Cutover to production enrichment services is a per-service `UPSTREAM_*` environment variable (`UPSTREAM_SEARCH`, `UPSTREAM_TRANSLATION`, …) — no rebuild, no file edit. The JSON contracts are frozen (`server.js` header), so no client rework either way.
- **Caller identity** — the SPA pod reads the authenticating front's identity headers. Names are configuration (`k8s/auth.yaml`), so moving between fronts is a ConfigMap edit. See §8.

7. CI validation (what "proven on a cluster" means here)

`package-validate` in `.github/workflows/angular-ci.yml`, on every push:

1. Build all six images.
2. trivy scan gating on CRITICAL (unfixed excluded).
3. kind cluster → `kubectl apply -k deploy/k8s` → rollout status on all six Deployments.
4. Smoke through the SPA proxy chain: healthz, bundle, search, translate, OCR, static fixture, the vendored icon font, and `/api/whoami` both with and without identity headers.
5. Redeploy the *same images* through `deploy/overlays/tls` with a generated certificate: all six pods on https, probes on HTTPS, edge-to-pod over TLS, and a check that the TLS port refuses plaintext.
6. Assemble the air-gap transfer bundle — after all of the above, so the media that crosses is provably what passed.

A green run is the deployable-artifact proof; the Actions log is the evidence record. What it does *not* prove is anything the target environment-specific: admission policy, registry gating, ingress, the real front, the enclave CA, or the model gateway.

8. Caller identity (proxy headers)

The enclave front (OIDC / mTLS) authenticates the user and forwards identity as request headers. The SPA pod reads them, exposes them at `GET /api/whoami`, shows the officer in the toolbar, and attributes their notes in the gold copy.

Header names are deployment-specific and therefore configuration, never code — `k8s/auth.yaml`:

Key	Default	Meaning
`AUTH_MODE`	`disabled`	`proxy-header` to require identity; `disabled` parses and displays it without enforcing
`AUTH_HEADER_ID`	`x-ain`	Unique caller id
`AUTH_HEADER_NAME`	`x-name`	Display name
`AUTH_HEADER_GROUPS`	`x-ismemberof`	Group membership
`AUTH_HEADER_ORG`	`x-org`	Organization
`AUTH_GROUP_SEPARATOR`	`;`	Characters that split the groups value
`AUTH_REQUIRED_GROUPS`	*(empty)*	Comma-separated; caller must hold ANY one. Empty ⇒ authenticate only

In `proxy-header` mode a request without identity gets `401`; one with the wrong groups gets `403`. `/healthz` is always exempt — a probe carries no identity, and gating it would restart the pod forever.

These headers are trusted absolutely. That is safe only where the pod cannot be reached except through the authenticating front, because anyone who can open a socket to it directly can assert any identity. Restricting ingress to the front is a target environment-side control these manifests cannot express — it is a deployment prerequisite. See `AIRGAP.md` §9.